



Tribhuvan University
Faculty of Humanities and Social Science

A PROPOSAL REPORT
ON
GAZE TRACKING SYSTEM

Submitted to
Department of Computer Application
Pascal National College

*In partial fulfillment of the requirements for the degree of Bachelors in Computer
Application*

Submitted by
Keshab Poudel | 6-2-1226-10-2021
April 2026

1. Introduction

Eye gaze is one of the most natural and expressive signals of human attention. The ability to determine where a person is looking at any given moment has applications spanning usability research, medical diagnostics, assistive technology, and human-computer interaction. Despite the sophistication of commercial eye-tracking hardware, such devices remain expensive and inaccessible to the majority of developers and researchers. The emergence of machine learning-based facial landmark detection libraries has made it practical to extract gaze information from a standard webcam, opening the door to accessible, low-cost gaze tracking for everyday computing environments.

This project, titled Gaze Tracking System, aims to address this opportunity by developing a complete end-to-end system that will capture eye gaze through an ordinary webcam, estimate where on the screen the user is looking, and visualize that gaze position in real time within a web browser. The system will be composed of three integrated layers: a Python computer vision service that will process video frames and extract gaze signals, a Django backend that will manage authentication, calibration profiles, and session data, and a React frontend that will display a gaze cursor and calibration interface.

The core of the gaze pipeline will be built on MediaPipe Face Mesh, which provides 478 precise facial landmarks per frame including refined iris landmarks. From these landmarks, a set of derived signals will be computed: iris offset relative to eye corners will yield a raw gaze direction vector, a PnP-based head pose estimate will remove the confounding effect of head rotation, polynomial regression trained during a 9-point calibration process will map the corrected gaze signal to actual screen coordinates, and an I-DT fixation detection algorithm will classify whether the user is maintaining a stable gaze or moving between regions.

The system is being developed as a final year BCA project at Tribhuvan University, Nepal, and is intended to demonstrate the integration of computer vision, real-time communication, and web technologies into a functioning research prototype.

2. Problem Statement

Existing eye-tracking solutions are either confined to expensive dedicated hardware or are closed, proprietary software with limited research utility. Students, developers, and researchers who wish to study gaze behavior or build gaze-aware applications face a significant barrier due to the cost and complexity of currently available tools. While commodity webcams are now capable of capturing sufficient facial detail for landmark-based gaze estimation, there is no simple, open, and fully integrated system that combines real-time gaze estimation with a web-based visualization and session logging interface.

Furthermore, raw gaze estimation from iris landmarks is inherently noisy and confounded by head rotation. Without a calibration step and a pose correction mechanism, gaze coordinates cannot be reliably mapped to actual screen positions across different users, camera placements, or screen sizes. This project will address these gaps by building a complete, calibrated, real-time gaze tracking pipeline accessible through a standard web browser, using only open-source libraries and standard consumer hardware.

3. Objectives

The main objective of this project is to design and develop a complete real-time gaze tracking system using a standard webcam and web technologies. The specific objectives are as follows:

- To capture real-time video from a standard webcam and extract 478 three-dimensional facial landmarks using MediaPipe Face Mesh.
- To compute a normalized gaze direction vector from iris position relative to eye corner landmarks and correct it for head pose using PnP-based rotation estimation.
- To implement a 9-point polynomial regression calibration procedure that will map the corrected 6-element gaze feature vector to accurate screen pixel coordinates.
- To detect fixations using an I-DT sliding window dispersion algorithm and stream gaze data to the web frontend over WebSocket at approximately 30 frames per second.
- To generate a spatial heatmap of gaze density for each session using Kernel Density Estimation (KDE)

4. Methodology

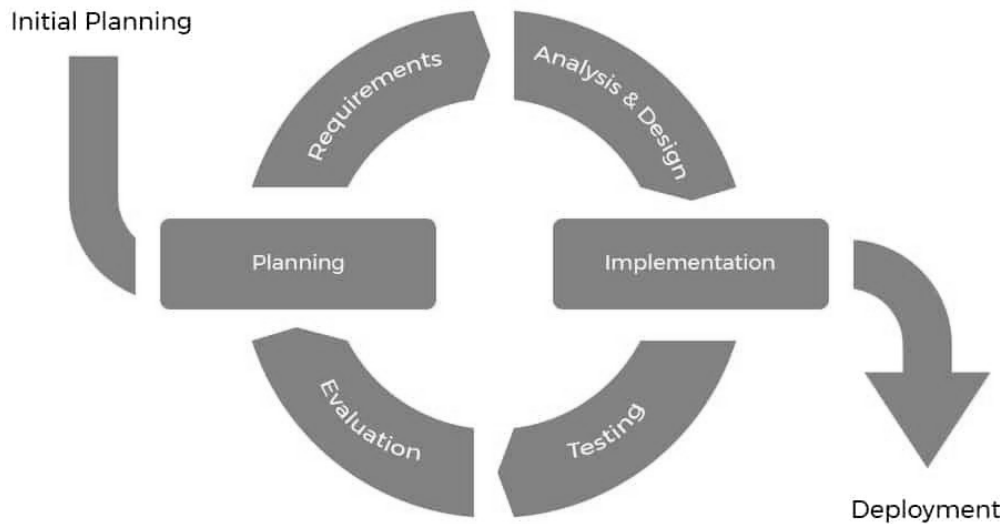


Figure 1: Iterative Development Methodology

The Gaze Tracking System will be developed following an iterative development methodology, building and validating each component of the pipeline before integrating it with the next. Development will proceed in four broad phases: backend infrastructure and authentication, CV pipeline integration and WebSocket streaming, frontend gaze visualization and calibration, and session logging and heatmap generation. This approach will allow each layer to be independently tested before system-level integration is performed.

a. Requirement Identification

i. Study of Existing System

Gaze tracking as a field of study dates back several decades, with early systems relying on infrared illumination and specialized cameras to detect the pupil and corneal reflection. These systems, while accurate, required controlled laboratory environments and expensive equipment. The shift toward appearance-based gaze estimation using standard cameras emerged as a practical alternative, leveraging advances in machine learning to extract gaze information from facial images without specialized hardware.

MediaPipe Face Mesh, released by Google, provides a real-time neural network-based solution for detecting 478 three-dimensional facial landmarks from a single RGB image. The inclusion of refined iris landmarks at indices 468 and 473 makes it directly applicable to gaze estimation by providing the iris center position. The Perspective-n-Point (PnP)

problem, solved using OpenCV's `solvePnP` function, is a classical computer vision technique for estimating the 3D pose of an object from a set of 2D-to-3D point correspondences, and will be used in this project to estimate the user's head orientation.

Polynomial regression is a well-understood supervised learning technique that extends linear regression by mapping input features to a higher-dimensional polynomial feature space before fitting a linear model. Applied to gaze calibration, it will allow the system to learn a non-linear mapping from the 6-element gaze feature vector to screen coordinates from a small set of calibration samples collected during the guided calibration procedure. The I-DT (Dispersion-Threshold Identification) algorithm is one of the most widely cited fixation detection methods in eye-tracking literature, classifying a sequence of gaze points as a fixation if they remain within a spatial dispersion threshold over a minimum time window.

Django Channels extends the Django web framework with support for WebSocket connections using an ASGI-compatible server. It will be used in this project to maintain a persistent bidirectional channel between the Python CV service and the React frontend. React, a JavaScript library for building user interfaces, will be used on the frontend to manage real-time state updates and render the gaze cursor with minimal latency.

ii. Literature Review

Krafka et al. in "Eye Tracking for Everyone" [1] demonstrated that appearance-based gaze estimation from smartphone cameras is feasible for consumer applications, achieving person-independent gaze estimation using a convolutional neural network trained on a large dataset. Their work established that low-cost camera-based gaze tracking is a viable research direction and highlighted the critical importance of per-user calibration in achieving usable accuracy — a finding that directly motivates the calibration module in this project.

Cheng et al. in their survey of appearance-based gaze estimation [2] reviewed multiple approaches to gaze vector computation including iris-based methods, pupil detection methods, and full-face regression models. The iris offset approach to be used in this project, while simpler than end-to-end learned approaches, is noted for its interpretability and low computational cost, making it well-suited for real-time applications on commodity hardware without GPU acceleration.

Salvucci and Goldberg's foundational work on fixation identification algorithms [3] evaluated several algorithms including I-DT and I-VT (velocity threshold). They found that I-DT performs well for identifying fixations in gaze data with moderate noise levels,

making it appropriate for webcam-based gaze signals which carry more noise than infrared systems. Their recommended parameters of a 150ms minimum window duration and a spatial dispersion threshold of approximately 1 degree of visual angle inform the parameter choices in this project's FixationDetector implementation.

Commercial tools such as Tobii Eye Tracker and GazeRecorder provide high-accuracy eye tracking but require proprietary hardware or paid subscriptions respectively. Open-source alternatives such as OpenGazer and GazeML exist but are either unmaintained or require GPU-accelerated inference, significantly limiting accessibility. This project will fill a gap by providing a fully integrated, lightweight, and open system suitable for a development and research context on standard consumer hardware.

iii. Requirement Analysis

1. Functional Requirements

The functional requirements define the specific behaviors and capabilities the Gaze Tracking System must provide.

Table 1: Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	The system shall allow users to register and log in using JWT-based authentication. Protected routes shall redirect unauthenticated users to the login page.
FR-02	9-Point Calibration	The system shall provide a guided 9-point on-screen calibration procedure where the user fixates on target dots while the CV service records gaze feature vectors.
FR-03	Real-Time Gaze Stream	The system shall stream gaze coordinates from the CV service to the browser over WebSocket at approximately 30 frames per second.
FR-04	Gaze Cursor Rendering	The system shall render a visible gaze cursor on the browser screen at the received gaze coordinates, updating position each frame.
FR-05	Face Loss Handling	The system shall hide the gaze cursor and display a status indicator when no face is detected in the webcam frame.
FR-06	Calibration Persistence	The system shall store polynomial regression calibration coefficients per user in the database and retrieve them automatically when a tracking session begins.
FR-07	Session Logging	The system shall log gaze sessions and individual gaze point coordinates to the database for retrospective analysis.
FR-08	Heatmap API	The system shall provide a REST API endpoint that aggregates stored gaze points for a session and returns KDE-smoothed density data for heatmap visualization.

2. Non-Functional Requirements

- **Performance:** The system shall process and stream gaze data at a minimum of 25 frames per second to ensure smooth cursor movement on the frontend.
- **Latency:** WebSocket message round-trip latency from CV service to browser shall remain below 100 milliseconds under normal local operating conditions.
- **Persistence:** The calibration model shall store polynomial regression coefficients in a compact JSON format in the database, serializable and deserializable without retraining.
- **Portability:** The frontend shall function correctly on any modern web browser without the installation of additional plugins or extensions.
- **Security:** User authentication tokens shall be stored securely and shall be included in WebSocket connection establishment for per-user channel isolation.
- **Reliability:** The system shall recover gracefully from camera disconnection or face loss without crashing the CV service process, and shall resume tracking automatically when the face re-enters the camera frame.

b. Feasibility Study

i. Technical

The Gaze Tracking System will use Python, Django, Django Channels, React, OpenCV, and MediaPipe — all of which are mature, well-documented, and freely available under open-source licenses. The hardware requirement is a standard webcam, which is present on most modern laptops including the development machine for this project. The WebSocket communication between the CV service and the Django backend, and between the Django backend and the React frontend, uses established protocols with reliable library support. The project is technically feasible on a single developer workstation without GPU acceleration.

ii. Operational

The Gaze Tracking System will be operable by any user with basic familiarity with web applications. The calibration procedure will be guided by the frontend interface with clear visual instructions. The only non-standard step required of the user is launching the CV service from a terminal before beginning a session, which is acceptable for the intended research and demonstration context of this project.

iii. Economic

The project will require no paid software, APIs, or hardware beyond a standard developer laptop with a built-in webcam. All libraries to be used are open-source with permissive licenses. A demonstration deployment can be hosted on a local machine at no cost. The project is fully economically feasible within a student project budget of zero additional expenditure.

c. High Level Design of System

i. Object Modelling using Class and Object Diagrams

The class diagram below represents the key classes in the Gaze Tracking System, their attributes, methods, and relationships. The User class will have a one-to-one relationship with CalibrationProfile (each user will have at most one stored calibration) and a one-to-many relationship with GazeSession (a user may have multiple tracked sessions). Each GazeSession will have a one-to-many relationship with GazePoint. The GazeCalibrationModel and FixationDetector classes will exist entirely within the CV service and will have no direct database representation; their state will either be serialized to CalibrationProfile or discarded at session end.

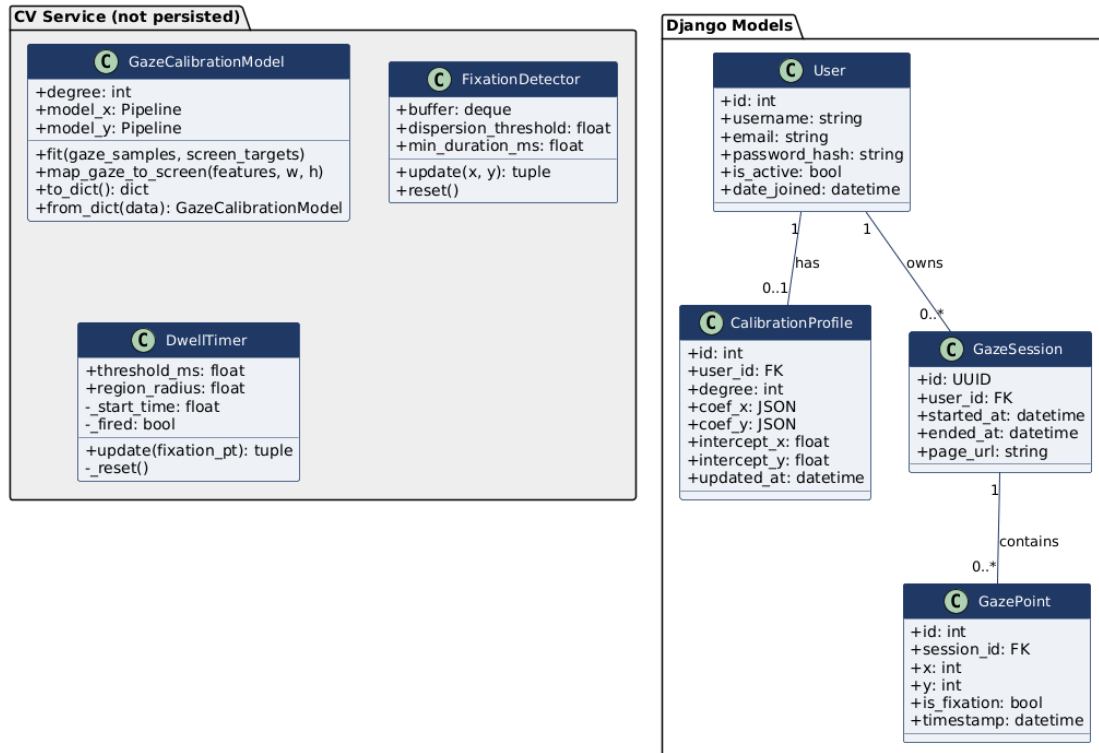


Figure 2: Class Diagram — Gaze Tracking System

ii. Dynamic Modeling using State and Sequence Diagrams

The sequence diagram below illustrates the message flow between the React browser frontend, the Django Channels WebSocket consumer, and the Python CV service during a complete gaze tracking session, from initial connection through calibration to live tracking.

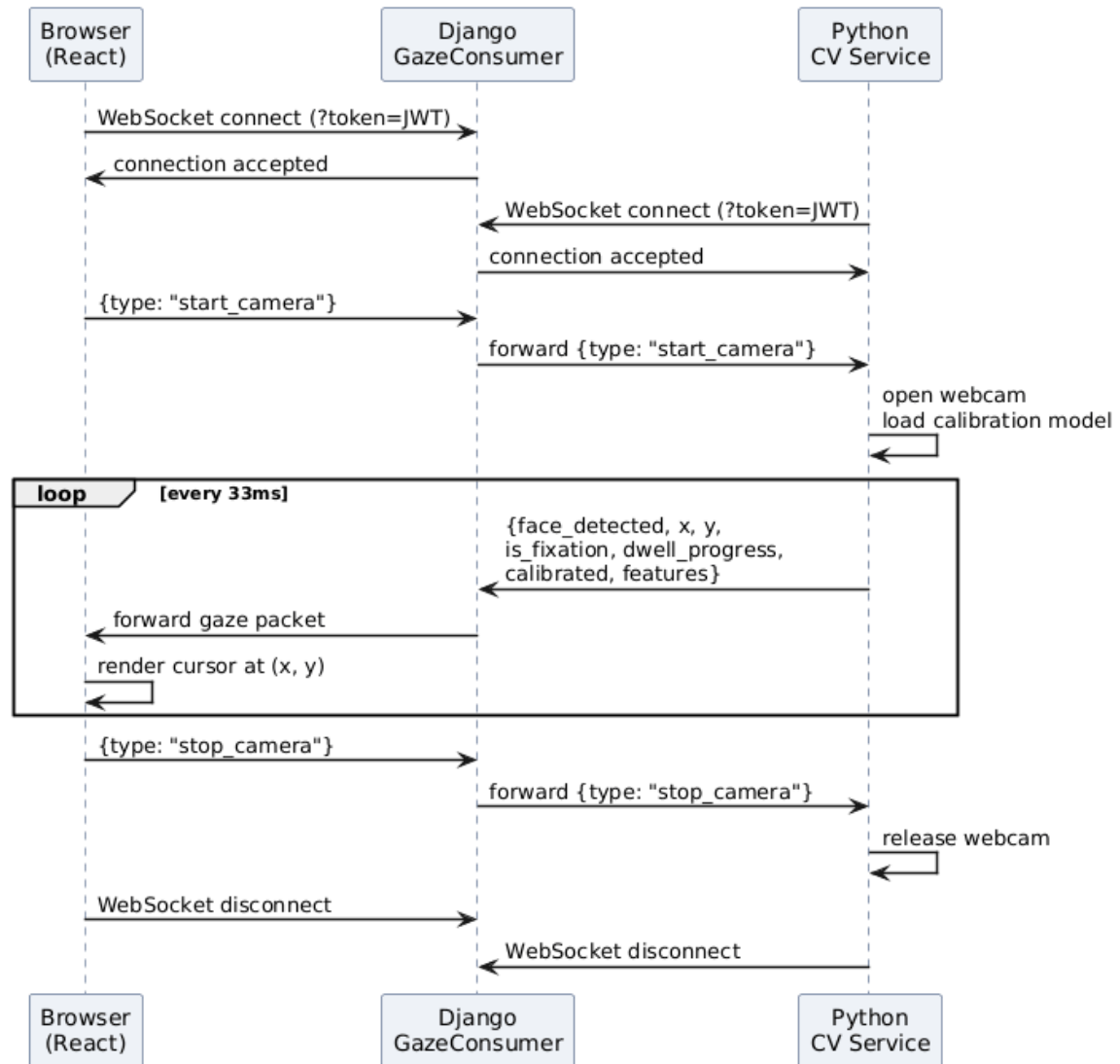


Figure 3: Sequence Diagram — Gaze Tracking Session

The state diagram below shows the lifecycle of the CV service process from startup to termination, illustrating how it transitions between the idle and active tracking states in response to control messages received over the WebSocket connection.

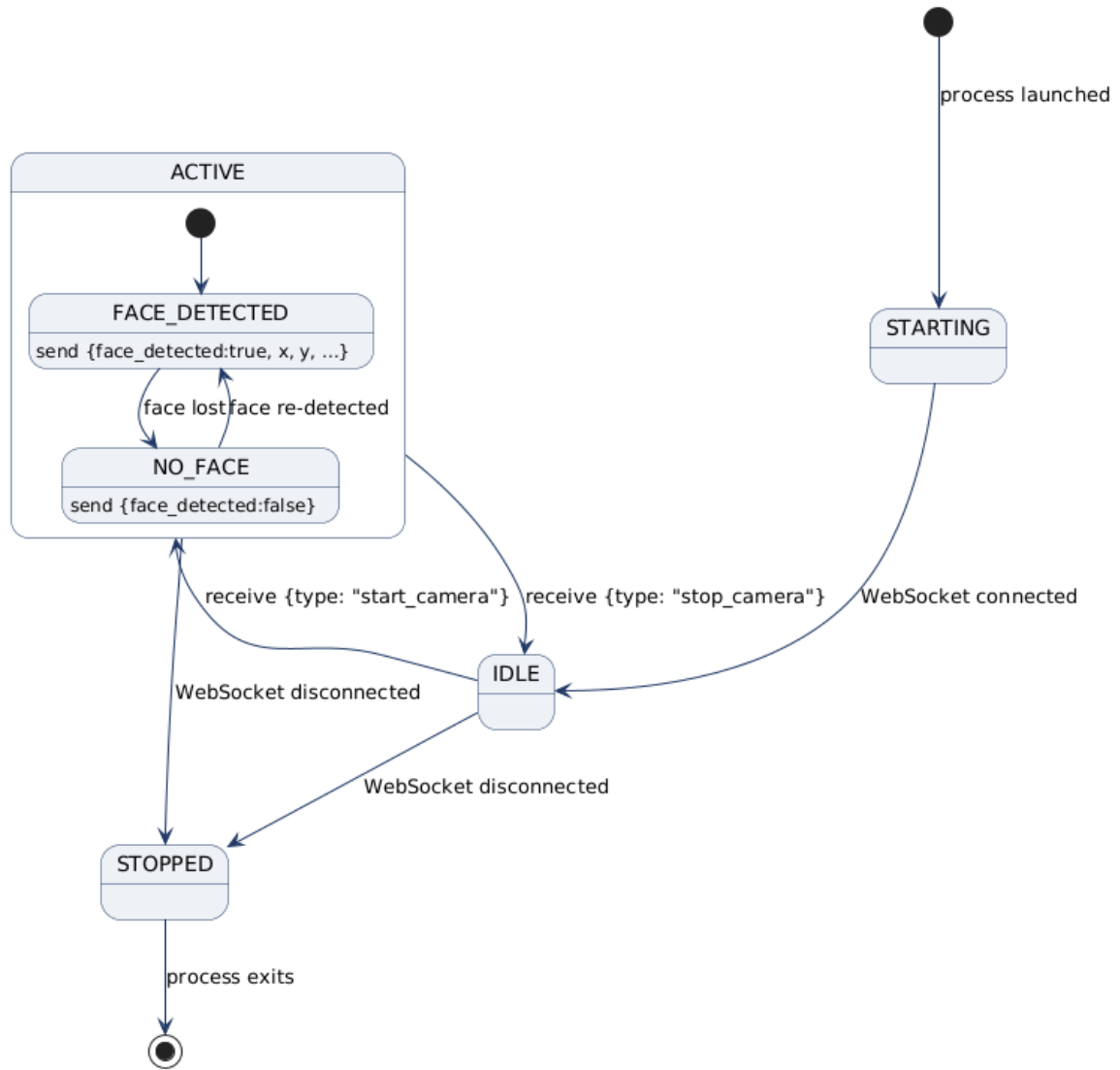


Figure 4: State Diagram — CV Service Lifecycle

iii. Process Modelling using Activity Diagrams

The activity diagram below describes the per-frame processing pipeline that the CV service will execute for each video frame captured from the webcam, from frame acquisition through gaze estimation to WebSocket transmission.

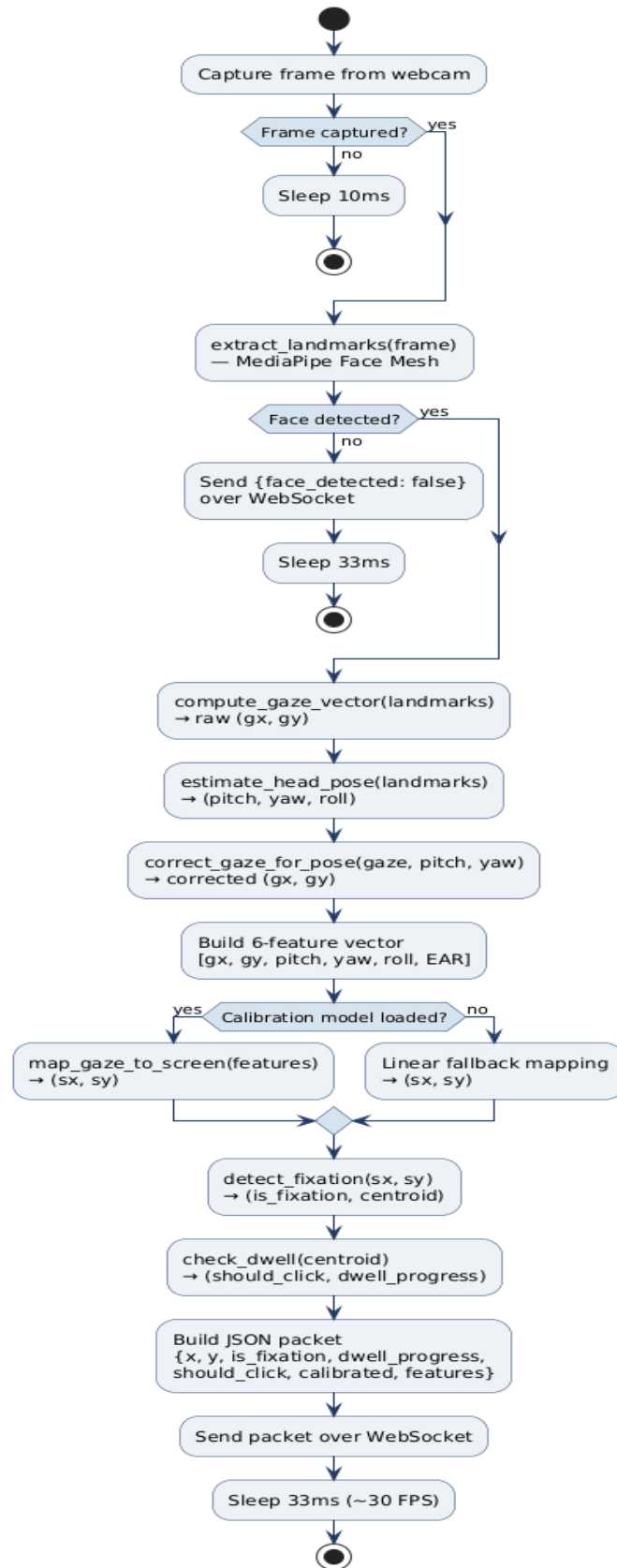


Figure 5: Activity Diagram — Per-Frame Gaze Processing

iv. Refinement of Diagrams

The use case diagram below identifies the primary interactions between the authenticated user and the Gaze Tracking System. Six use cases cover the complete functional scope of the system as defined in the requirements.

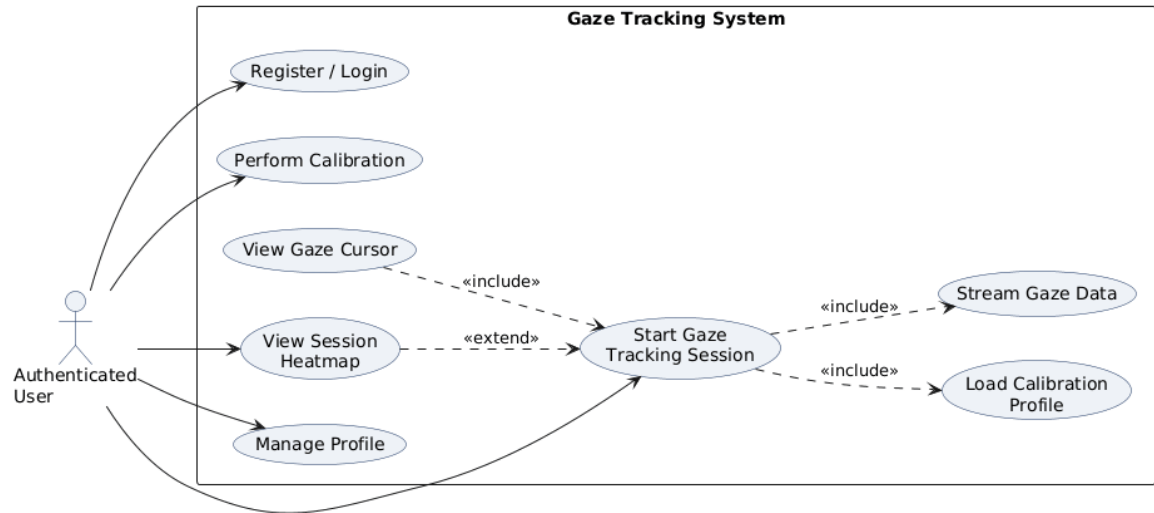


Figure 6: Use Case Diagram

The Entity-Relationship diagram below shows the database schema of the Gaze Tracking System with entities, attributes, primary and foreign key relationships.

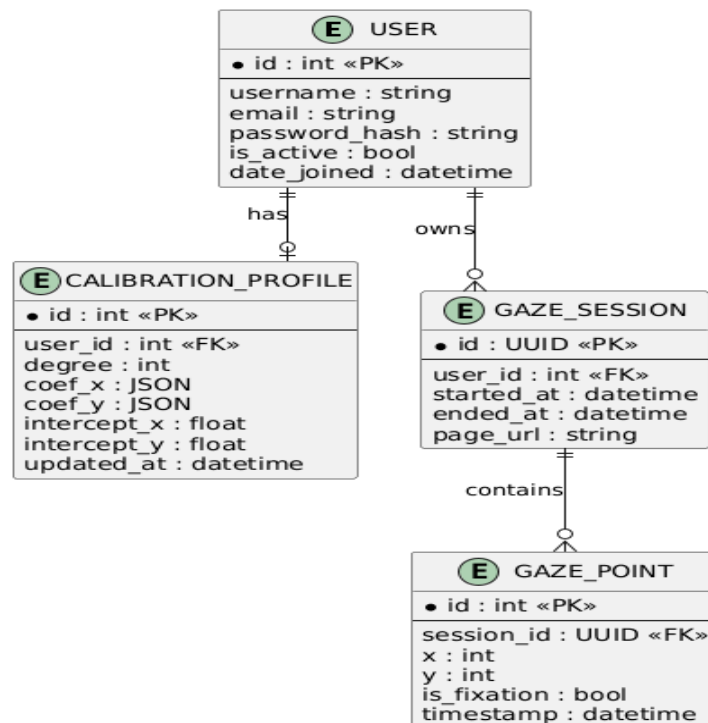


Figure 7: ER Diagram — Database Schema

v. Component Diagram

The component diagram below illustrates the three major runtime components of the Gaze Tracking System — the React frontend, the Django backend, and the Python CV service — along with the communication interfaces between them.

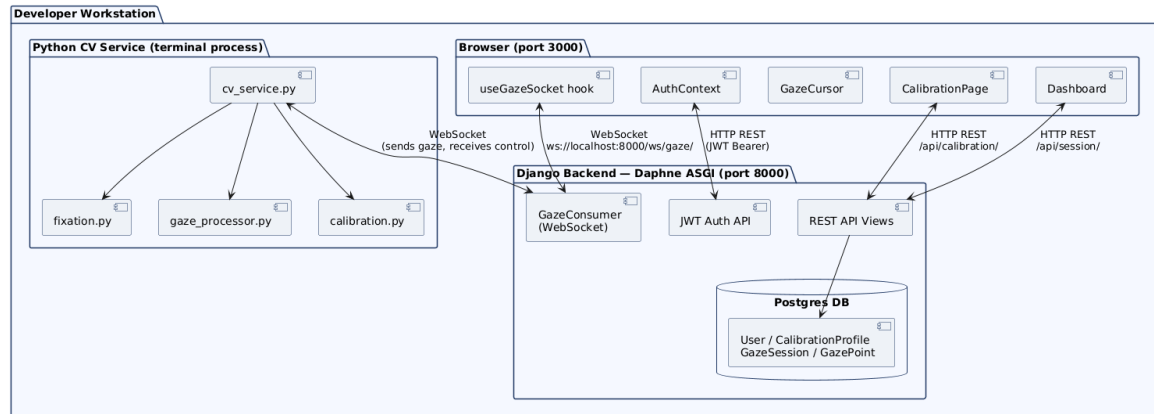


Figure 8: Component Diagram

vi. Deployment Diagram

The deployment diagram below shows how the three components of the Gaze Tracking System will be deployed across execution environments on the user's local workstation during development and demonstration.

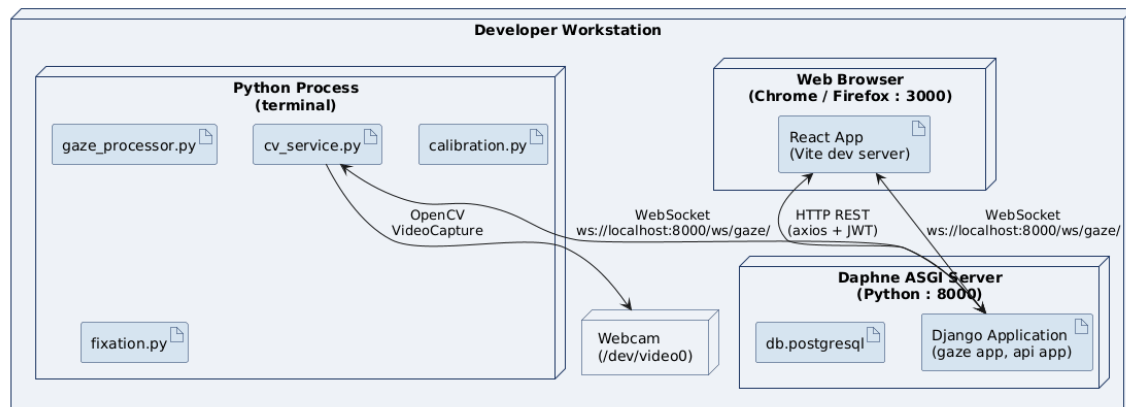


Figure 9: Deployment Diagram

5. Gantt Chart

The gantt chart presents the planned project timeline across 16 weeks of the eighth semester. Blue cells represent active development phases and red cells represent evaluation milestones.

Table 2 : Project Schedule — Gantt Chart

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Requirements & System Analysis	Development phase	Development phase	Development phase													
Literature Review		Development phase	Development phase	Development phase												
Feasibility Study			Development phase	Development phase												
Proposal Defense				Milestone												
Backend & Auth Setup					Development phase	Development phase										
Django Channels / WebSocket						Development phase	Development phase	Development phase								
CV Pipeline Development						Development phase	Development phase	Development phase	Development phase							
Calibration Module							Development phase	Development phase	Development phase	Development phase						
React Frontend & Gaze Cursor										Development phase	Development phase	Development phase	Development phase			
Session Logging & Heatmap												Development phase	Development phase	Development phase		
Testing & Debugging													Development phase	Development phase	Development phase	
Midterm Defense												Milestone				
Report Writing & Finalization														Development phase	Development phase	Development phase
Final Defense																Milestone
Development phase ■ Milestone ■																

6. Expected Outcome

Upon successful completion, the Gaze Tracking System will be a fully functional, real-time gaze tracking application accessible through a standard web browser. The system will capture gaze from a standard webcam, apply head-pose-corrected polynomial regression calibration, and render a smooth gaze cursor on any page within the application. Users will be able to register an account, perform a guided 9-point calibration, and immediately observe their gaze position reflected as a cursor moving in response to their eye movements.

Gaze sessions will be logged to the database, enabling retrospective analysis of user attention distribution. A heatmap endpoint will aggregate logged gaze points using Kernel Density Estimation and return a spatial density grid suitable for visualization as an

attention overlay. This will support usability research and educational demonstrations of gaze-based interaction.

The project will demonstrate the feasibility of accurate, low-cost, browser-integrated gaze tracking using only a standard webcam and open-source software on commodity hardware. It will serve as a working research prototype that could be extended in future work to support gesture-based interaction, multi-user sessions, remote browser deployment, or integration with deep learning-based gaze estimation models for improved accuracy.

7. References

- [1] C. Krafska et al., "Eye Tracking for Everyone," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, USA, 2016, pp. 2176–2184.
- [2] Y. Cheng, H. Wang, Y. Bao and F. Lu, "Appearance-Based Gaze Estimation With Deep Learning: A Review and Benchmark," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 46, no. 1, pp. 1–20, 2023.
- [3] D. D. Salvucci and J. H. Goldberg, "Identifying Fixations and Saccades in Eye-Tracking Protocols," in Proc. ACM Symposium on Eye Tracking Research and Applications (ETRA), Palm Beach Gardens, USA, 2000, pp. 71–78.
- [4] Y. Kartynnik, A. Ablavatski, I. Grishchenko and M. Grundmann, "Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs," arXiv preprint arXiv:1907.06724, 2019.
- [5] Google LLC, "MediaPipe Face Mesh," [Online]. Available: <https://mediapipe.dev>. [Accessed: Apr. 2026].
- [6] Django Software Foundation, "Django Channels Documentation," [Online]. Available: <https://channels.readthedocs.io>. [Accessed: Apr. 2026].
- [7] A. T. Duchowski, Eye Tracking Methodology: Theory and Practice, 3rd ed. London, UK: Springer, 2017.